

LAB: Line Detection (Lane Keeping Assist System)

Date: 2026-04-06

Author: lee jeayong(22000561)

Introduction

1. Objective

The goal of this lab is to develop a machine vision system capable of detecting driving lanes on a road to assist in Lane Keeping. We are tasked to detect the left and right lanes, highlight the drivable area, and pinpoint the vanishing point to project steering guidelines.

The system will perform the following tasks:

- Extract line segments from a given road image using Canny Edge and Hough Transform.
- Classify lines into Left and Right lanes based on their slopes.
- Calculate the optimal single lane representation and the road's vanishing point.
- Visualize the lanes, drivable path (semi-transparent fill), and the vanishing point with intermediate step visualizations.

2. Preparation

Software Configuration

- OpenCV, C++ (Visual Studio)

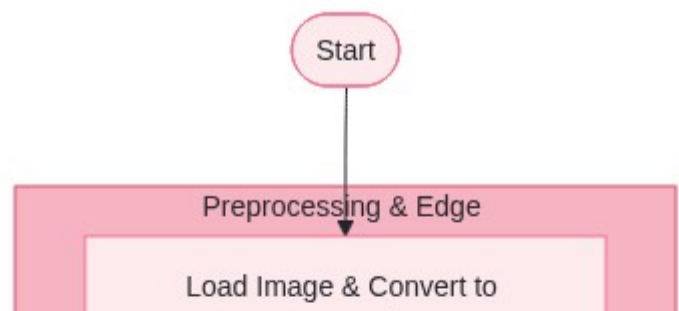
Dataset

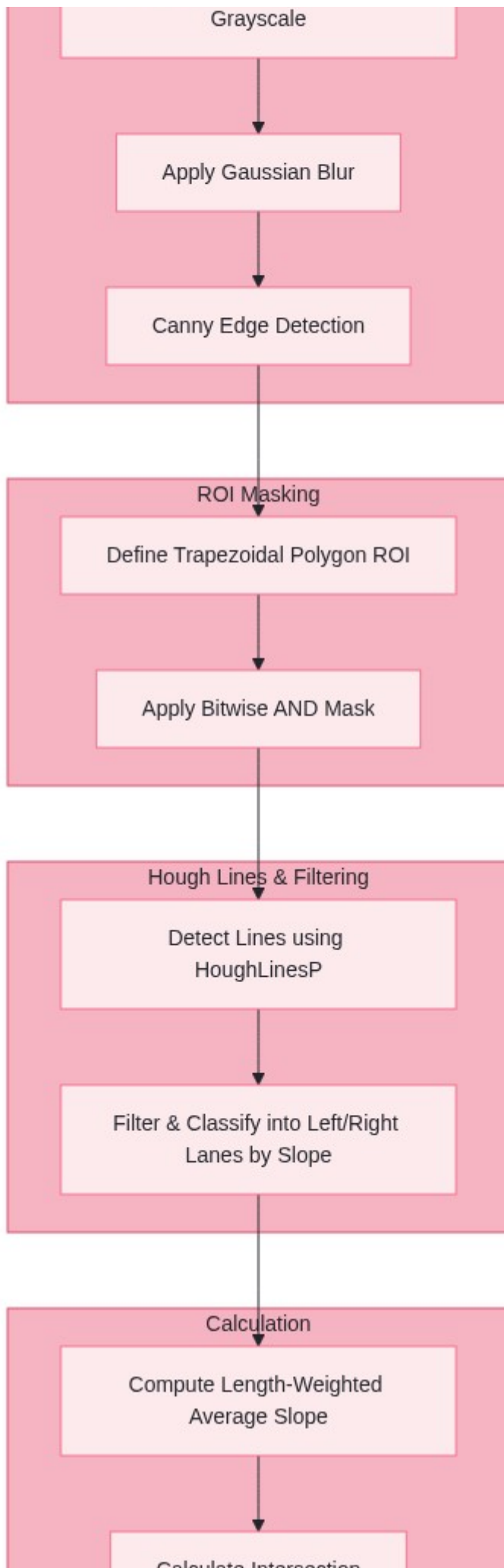
- `Lane_center.jpg` (Ego-vehicle in the center of the lane)
- `Lane_changing.jpg` (Ego-vehicle changing lanes)

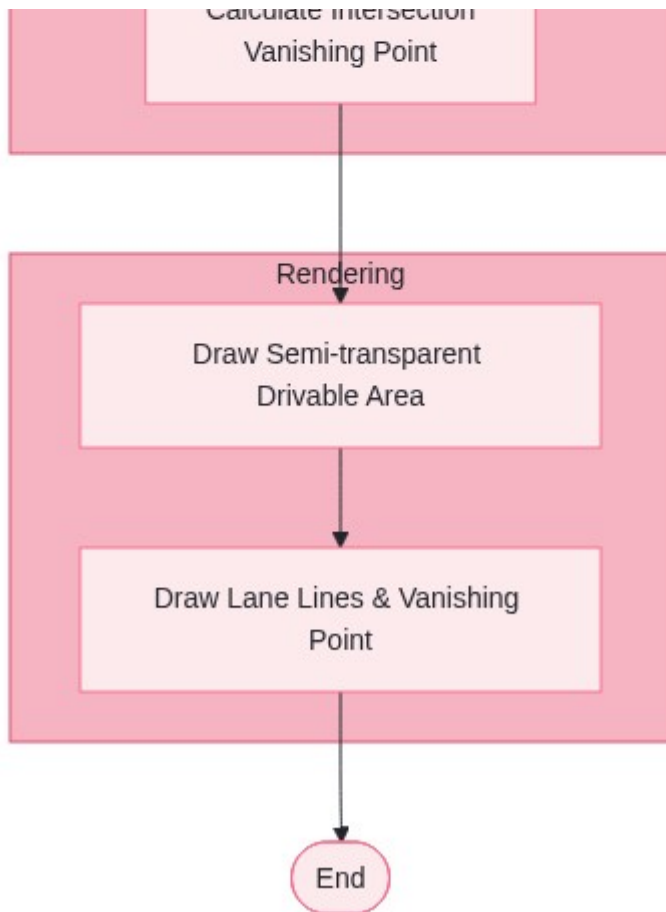
Algorithm

1. Overview

The algorithm follows sequential steps to identify the lane logic.







2. Procedure & Step-by-Step Visualization

Step 1: Preprocessing & Edge Detection

To eliminate superficial noises from the road surface, **GaussianBlur** is applied initially.

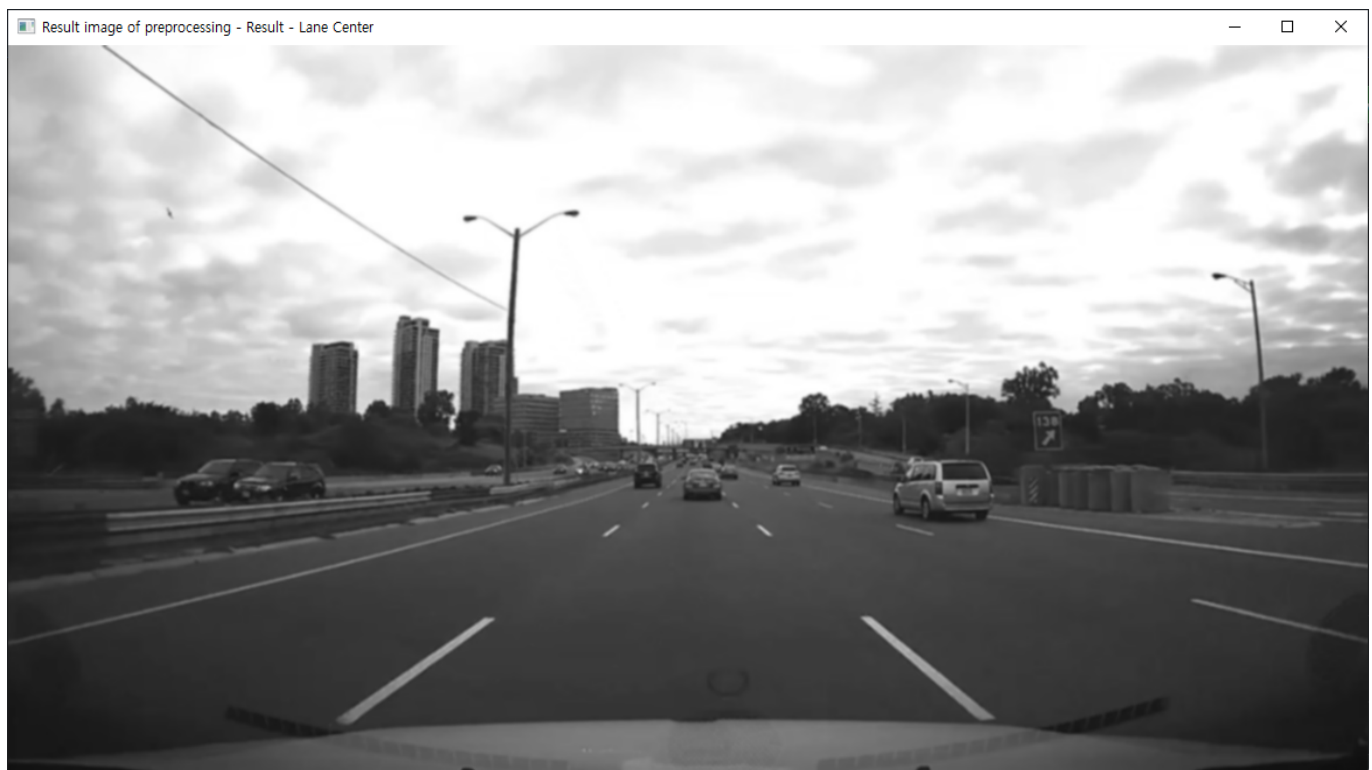


Figure 1: Preprocessing (Blur) for Lane_center.jpg



Figure 2: Preprocessing (Blur) for Lane_changing.jpg

Step 2: Canny Edge & ROI Masking

Canny edge detection is performed to identify strong structural lines. Then, a trapezoidal ROI polygon is defined strictly framing the lower road area (avoiding the upper horizon). `fillConvexPoly` generates a logical mask, and bitwise AND operation guarantees that we only analyze edges located on the actual road segment.

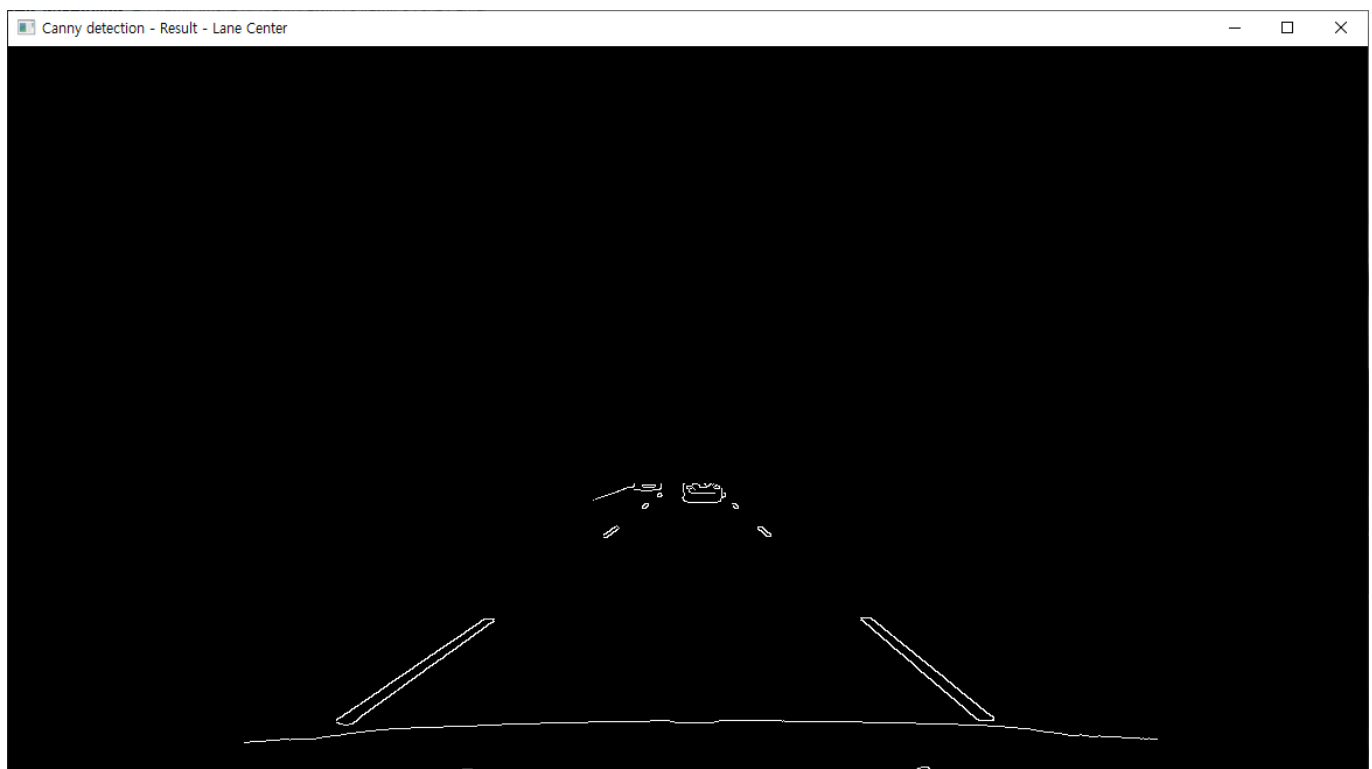


Figure 3: Canny Edge with ROI mask for Lane_center.jpg



Figure 4: Canny Edge with ROI mask for Lane_changing.jpg

Step 3: Hough Lines & Filtering

HoughLinesP processes the masked edges. Multiple small lines belonging to lane markings and road borders are generated. The raw lines are drawn (Blue for left, Green for right) to display the extraction performance before computing the weighted average. Vertical and extreme horizontal lines are explicitly ignored based on their slopes; this is done to robustly filter out irrelevant road structures (such as pedestrian crosswalks) and to prevent fatal zero-division errors in line equation calculations, keeping only the lines that genuinely look like driving lanes.

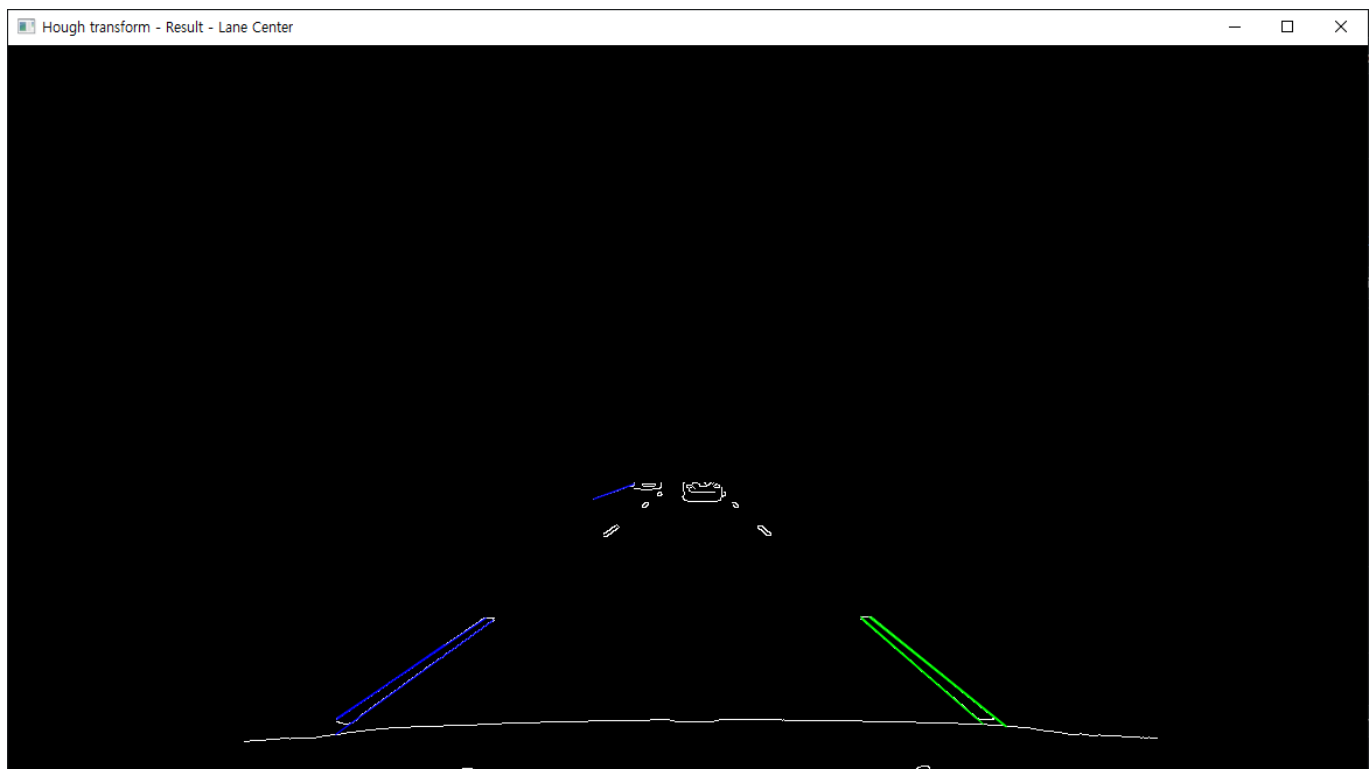


Figure 5: Hough Transform raw lines for Lane_center.jpg



Figure 6: Hough Transform raw lines for Lane_changing.jpg

Step 4: Final Rendering & Vanishing Point

A length-weighted average calculates the final dominant lines, giving a robust estimation for the vanishing point. To enhance driver visualization, `addWeighted` blends an extrapolated green area between the bottom corners and the vanishing point (representing the safety drivable path). Left/Right solid lane boundaries and the vanishing marker are superimposed.



Figure 7: Final Result for Lane_center.jpg

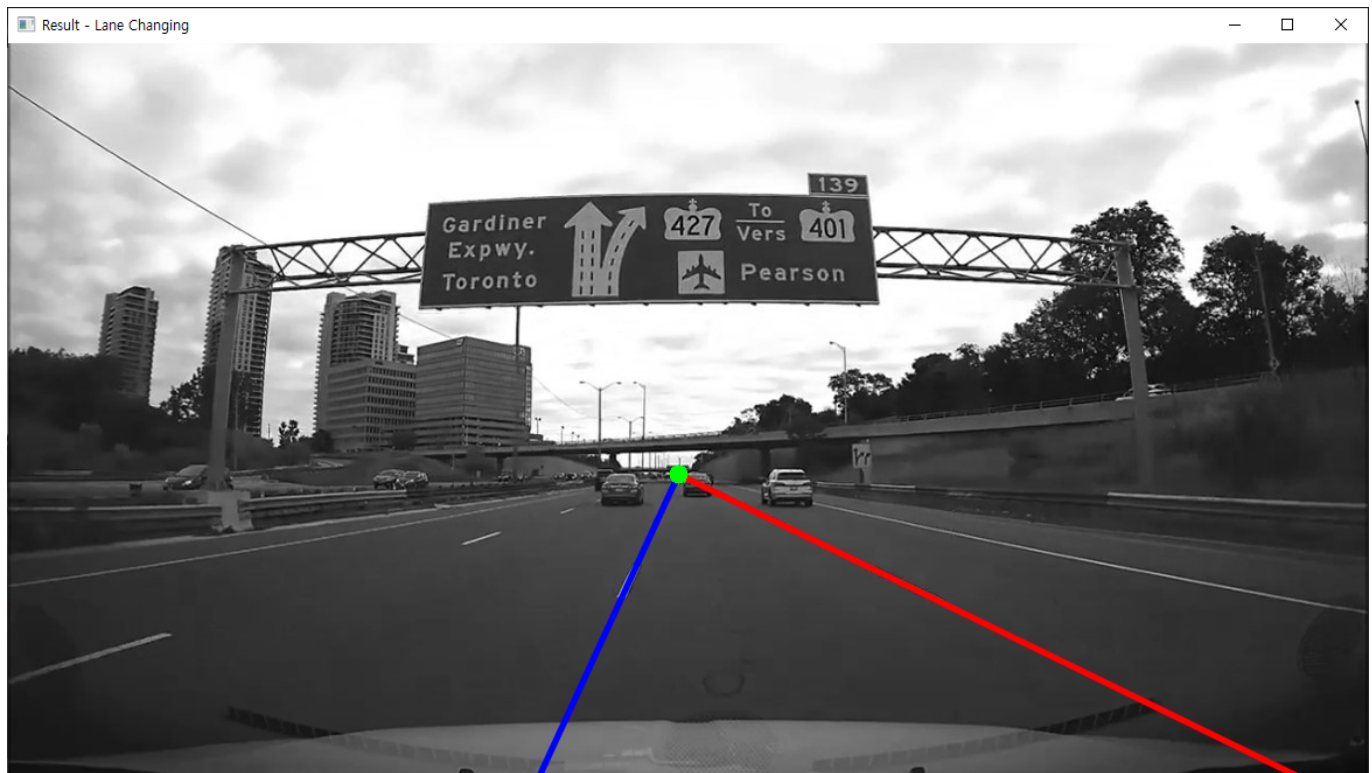


Figure 8: Final Result for Lane_changing.jpg

Conclusion

The machine vision project successfully maps standard road lane boundaries and calculates predictive markers (vanishing points). The implementation of sequential visualization via `imshow` also aids profoundly in debugging the various stages (Blur -> Canny ROI -> Hough raw vectors -> Final lane overlay). The length-weighted averaging method effectively rejects small anomalous markings or bumps on the road since long continuous lanes dominate the weight calculation.

Appendix

DLIP_Assignment_Line_Detection.cpp

```
#include "opencv2/imgcodecs.hpp"
#include "opencv2/highgui.hpp"
#include "opencv2/imgproc.hpp"
#include <iostream>
#include <vector>

using namespace cv;
using namespace std;

// Function to process the lane image pipeline
void processLaneImage(Mat& src, const string& window_name);

int main(int argc, char** argv)
{
```

```

// Set image paths
const char* filename_center = "../../Image/Lane_center.jpg";
const char* filename_changing = "../../Image/Lane_changing.jpg";

Mat src_center = imread(filename_center, IMREAD_GRAYSCALE);
Mat src_changing = imread(filename_changing, IMREAD_GRAYSCALE);

if (src_center.empty() || src_changing.empty()) {
    printf("Error: Cannot load one or both images.\n");
    return -1;
}

processLaneImage(src_center, "Result - Lane Center");
processLaneImage(src_changing, "Result - Lane Changing");

waitKey(0);
return 0;
}

// Function to process the lane image pipeline
void processLaneImage(Mat& src, const string& window_name) {
    Mat output;
    // Convert to BGR for color drawing
    cvtColor(src, output, COLOR_GRAY2BGR);

    // 1. Filtering (Blur) - Remove noise
    Mat blurred;
    GaussianBlur(src, blurred, Size(5, 5), 0);
    // [STEP 1 시각화] 전처리 결과 출력
    imshow("Result image of preprocessing - " + window_name, blurred);

    // 2. Canny Edge Detection
    Mat edges;
    Canny(blurred, edges, 50, 150);

    // 3. Apply polygonal (trapezoid) ROI mask
    Mat mask = Mat::zeros(edges.size(), edges.type());
    int height = edges.rows;
    int width = edges.cols;

    // Define ROI vertices (proportional to image size)
    Point pts_roi[4] = {
        Point(width * 0.1, height), // Bottom-left
        Point(width * 0.45, height * 0.6), // Top-left (near vanishing point)
        Point(width * 0.55, height * 0.6), // Top-right (near vanishing point)
        Point(width * 0.95, height) // Bottom-right
    };

    // Fill the mask interior with white (255)
    fillConvexPoly(mask, pts_roi, 4, Scalar(255));

    // Bitwise AND to keep edges only within ROI
    Mat masked_edges;
    bitwise_and(edges, mask, masked_edges);

```

```

// [STEP 2 시각화] Canny Edge (ROI 적용 후) 결과 출력
imshow("Canny detection - " + window_name, masked_edges);

// 4. Hough Line Detection
vector<Vec4i> lines;
HoughLinesP(masked_edges, lines, 1, CV_PI / 180, 30, 20, 20);

// [STEP 3 시각화 준비] Hough Transform 로우(raw) 라인을 그릴 빈 캔버스 준비
Mat hough_display;
cvtColor(masked_edges, hough_display, COLOR_GRAY2BGR); // 엣지 픽셀 위에 선분들
을 그리기 위해 변환

// 5. Classify Left/Right lanes and compute length-based weighted average
double left_m = 0, left_b = 0, left_weight = 0;
double right_m = 0, right_b = 0, right_weight = 0;

for (size_t i = 0; i < lines.size(); i++) {
    Vec4i l = lines[i];
    double x1 = l[0], y1 = l[1], x2 = l[2], y2 = l[3];

    if (x1 == x2) continue; // Skip vertical lines to avoid zero-division
error

    // Calculate line equation y = mx + b
    double m = (y2 - y1) / (x2 - x1);
    double b = y1 - m * x1;
    // Calculate line segment length
    double length = sqrt(pow(y2 - y1, 2) + pow(x2 - x1, 2));

    // OpenCV coordinates: y increases downwards
    // Left lane: x increases as y decreases -> negative slope
    if (m < -0.3 && m > -2.5) {
        left_m += m * length;
        left_b += b * length;
        left_weight += length;

        // [STEP 3] 좌측 원시 예측 선분 그리기 (파란색)
        line(hough_display, Point(x1, y1), Point(x2, y2), Scalar(255, 0, 0),
1, LINE_AA);
    }
    // Right lane: x increases as y increases -> positive slope
    else if (m > 0.3 && m < 2.5) {
        right_m += m * length;
        right_b += b * length;
        right_weight += length;

        // [STEP 3] 우측 원시 예측 선분 그리기 (초록색)
        line(hough_display, Point(x1, y1), Point(x2, y2), Scalar(0, 255, 0),
1, LINE_AA);
    }
}

// [STEP 3 시각화] 허프 변환으로 추출된 기초 단위의 선분들 모두 띄우기 (예시 이미지
와 동일)

```

```

imshow("Hough transform - " + window_name, hough_display);

bool has_left = left_weight > 0;
bool has_right = right_weight > 0;

// -----
// 6. Calculate the intersection (vanishing point) first
// -----
Point v_point(0, 0);
bool valid_v_point = false;

if (has_left && has_right) {
    // Finalize weighted average for slope and intercept
    left_m /= left_weight;
    left_b /= left_weight;
    right_m /= right_weight;
    right_b /= right_weight;

    // Calculate x, y coordinates of the intersection
    double x_v = (left_b - right_b) / (right_m - left_m);
    double y_v = left_m * x_v + left_b;

    // Check if intersection is within the screen bounds
    if (x_v > 0 && x_v < width && y_v > 0 && y_v < height) {
        v_point = Point(cvRound(x_v), cvRound(y_v));
        valid_v_point = true;
    }
}
else {
    // Fallback weights if only one side is detected
    if (has_left) {
        left_m /= left_weight;
        left_b /= left_weight;
    }
    if (has_right) {
        right_m /= right_weight;
        right_b /= right_weight;
    }
}

// -----
// 7. Draw semi-transparent lane fill, lines, and vanishing point
// -----

// Calculate rendering coordinates
int bottom_y = height;
// Use intersection point for top_y if valid, else use 60% of height
int top_y = valid_v_point ? v_point.y : cvRound(height * 0.6);

Point left_p1, left_p2, right_p1, right_p2;

if (has_left) {
    left_p1 = Point(cvRound((bottom_y - left_b) / left_m), bottom_y);
    left_p2 = Point(cvRound((top_y - left_b) / left_m), top_y);
}

```

```

    }
    if (has_right) {
        right_p1 = Point(cvRound((bottom_y - right_b) / right_m), bottom_y);
        right_p2 = Point(cvRound((top_y - right_b) / right_m), top_y);
    }

    // --- Draw semi-transparent fill between lanes (Draw this first) ---
    if (has_left && has_right) {
        // Temporary mask for transparency overlay
        Mat overlay = Mat::zeros(output.size(), output.type());

        // Define polygon vertices (fill between bottom and vanishing point)
        vector<Point> pts_fill;
        if (valid_v_point) {
            // Perfect triangle if vanishing point exists
            pts_fill = { left_p1, right_p1, v_point };
        }
        else {
            // Trapezoid fallback if no vanishing point
            pts_fill = { left_p1, right_p1, right_p2, left_p2 };
        }

        // Fill overlay with light green
        fillPoly(overlay, std::vector<std::vector<Point>>{pts_fill}, Scalar(0,
255, 0));

        // Blend original image and overlay (30% opacity)
        addWeighted(overlay, 0.3, output, 1.0, 0, output);
    }

    // --- Draw lanes (Draw after fill so lines render on top) ---
    // Draw averaged Left lane
    if (has_left) {
        line(output, left_p1, left_p2, Scalar(255, 0, 0), 4, LINE_AA); // Blue
    }

    // Draw averaged Right lane
    if (has_right) {
        line(output, right_p1, right_p2, Scalar(0, 0, 255), 4, LINE_AA); // Red
    }

    // Draw vanishing point markers
    if (valid_v_point) {
        // Draw vertical yellow drop line from vanishing point to bottom
        line(output, v_point, Point(v_point.x, height), Scalar(0, 255, 255), 2,
LINE_AA);

        // Draw vanishing point (Green circle)
        circle(output, v_point, 8, Scalar(0, 255, 0), -1);
    }

    // Display rendering result
    imshow(window_name, output);
}

```

